

# GitHub Actions

*A practitioner's guide – Hope this will helpful for all beginner*

*Based on real production experience across CI/CD pipelines, AKS, and DevOps toolchains*

## Before We Start — How to Read This Guide

---

I'm not going to teach you GitHub Actions the way a YouTube course does. We're not going to do hello world for 3 chapters before we touch anything real.

You already write production YAML — I've seen your CI pipeline. You already know enough to break and fix things. So here's the deal: I'm going to explain GitHub Actions the way I'd explain it to a new team member sitting next to me. Real context. Real mistakes we've all made. Real reasons behind every decision.

### How this guide is structured

We go from how GitHub Actions actually works inside, to how to write it, debug it, secure it, and optimize it. By the end you'll be able to read any workflow file in any repo and know exactly what it does and why.

## Chapter 1 — What is GitHub Actions, Really?

---

Most people explain GitHub Actions as 'a CI/CD tool'. That's like explaining a kitchen knife as 'a food tool'. Technically correct, completely useless.

Here's the real explanation:

GitHub Actions is an event-driven automation platform that runs code in response to things that happen in your GitHub repository.

That's it. Events trigger workflows. Workflows run jobs. Jobs run steps. Steps run commands or pre-built actions.

### The Mental Model That Changes Everything

Think of it this way. Your GitHub repo is a system. Things happen in that system — someone pushes code, opens a PR, creates a release, schedules a cron. GitHub Actions lets you say:

#### The core idea

"When THIS happens in my repo → run THESE commands on a machine somewhere"

That machine is called a runner. GitHub provides free Ubuntu/Windows/Mac runners, or you can bring your own (self-hosted runners).

### How GitHub Actions Executes — Step by Step

| Step                  | What happens                                          | Where               |
|-----------------------|-------------------------------------------------------|---------------------|
| 1. Event fires        | Push to main, PR opened, cron triggers                | GitHub servers      |
| 2. Workflow picked up | GitHub finds matching .yml file in .github/workflows/ | GitHub servers      |
| 3. Runner provisioned | Fresh VM spun up (ubuntu-latest = ~20 seconds)        | GitHub-hosted cloud |
| 4. Repo checked out   | Your code is cloned onto the runner                   | Runner VM           |
| 5. Steps execute      | Commands and actions run in sequence                  | Runner VM           |
| 6. Runner destroyed   | VM is deleted — nothing persists between runs         | Gone forever        |

### Critical insight

Every workflow run starts on a BRAND NEW machine. Nothing from the previous run exists. No files, no env vars, no Docker layers (unless you use caching). This is why you always need checkout, setup-node, login steps every single time.

## Chapter 2 — The Anatomy of a Workflow File

---

A workflow file lives at `.github/workflows/anything.yml`. You can have as many as you want. Each file is one workflow.

### The Skeleton

```
name: My Pipeline           # What you see in the GitHub Actions tab

on:                         # TRIGGER — when does this run?
  push:
    branches: [ main ]

env:                        # Global variables (available to all jobs)
  IMAGE_NAME: myapp

jobs:                       # One or more jobs
  job-id:                   # You name this — used for dependencies
    name: Friendly Name    # What shows in the UI
    runs-on: ubuntu-latest # Which machine to use
    steps:                 # List of tasks inside this job
      - name: Step name
        run: echo 'hello'  # Shell command
```

### The Four Top-Level Keys You Must Know

| Key  | Purpose                         | Required?              |
|------|---------------------------------|------------------------|
| name | Label shown in GitHub UI        | No (but always add it) |
| on   | What triggers this workflow     | YES                    |
| env  | Variables available to all jobs | No                     |
| jobs | The actual work to do           | YES                    |

### Triggers — The 'on' Block

This is the most important decision you make in a workflow — when should it run?

```
on:
  push:                       # Every push to these branches
    branches: [ main, develop ]

  pull_request:               # Every PR targeting these branches
    branches: [ main ]
```

```
schedule:                                # Cron — runs at fixed times
  - cron: '0 2 * * *'                   # Every day at 2 AM UTC

workflow_dispatch:                       # Manual run button in GitHub UI
  inputs:
    environment:
      description: 'Deploy to'
      required: true
      default: 'staging'

release:                                 # When a GitHub release is published
  types: [ published ]
```

### Real-world tip

In production pipelines, push triggers builds and tests. pull\_request triggers tests + security scans. release or workflow\_dispatch triggers deployments. Don't deploy on every push — it's chaos.

## Chapter 3 — Jobs, Steps, and the Difference Between 'run' and 'uses'

### Jobs

Jobs run IN PARALLEL by default. They each get their own fresh VM. They don't share files or environment.

```
jobs:
  test:                                # Job 1 — runs in parallel with 'lint'
    runs-on: ubuntu-latest
    steps: [ ... ]

  lint:                                # Job 2 — runs at the same time as 'test'
    runs-on: ubuntu-latest
    steps: [ ... ]

  deploy:
    needs: [test, lint]               # Job 3 — waits for BOTH to pass
    runs-on: ubuntu-latest
    steps: [ ... ]
```

#### The 'needs' keyword

This is how you control job order. 'needs: test' means this job won't start until 'test' passes. If 'test' fails, this job is skipped. Use it to gate deployments behind passing tests.

### Steps — run vs uses

Inside a job, you have steps. Every step is either a 'run' (your own shell command) or a 'uses' (someone else's pre-built action).

| Type | Syntax                    | When to use                                       |
|------|---------------------------|---------------------------------------------------|
| run  | run: echo hello           | Running shell commands, scripts, docker commands  |
| uses | uses: actions/checkout@v4 | Reusing pre-built actions from GitHub Marketplace |

```
steps:
  # 'uses' — runs someone else's packaged action
  - name: Checkout code
    uses: actions/checkout@v4

  # 'run' — your own shell commands
  - name: Build the app
```

```

run: |
  npm install
  npm run build

# 'run' with working directory
- name: Run tests
  working-directory: app/
  run: npm test

# 'uses' with 'with' for inputs
- name: Login to DockerHub
  uses: docker/login-action@v3
  with:
    username: ${ secrets.DOCKERHUB_USERNAME }
    password: ${ secrets.DOCKERHUB_TOKEN }

```

## Understanding 'uses: owner/repo@version'

When you write 'uses: docker/login-action@v3' — GitHub goes to [github.com/docker/login-action](https://github.com/docker/login-action), checks out the v3 tag, and runs that action's code on your runner. You can literally visit that URL and read the source.

| Version style     | Example  | Risk level                                      |
|-------------------|----------|-------------------------------------------------|
| Major version tag | @v4      | Low — safe, gets bug fixes automatically        |
| Exact version     | @v4.1.0  | Lowest — completely frozen, most stable         |
| Branch            | @master  | HIGH — can break without warning, avoid in prod |
| Commit SHA        | @abc1234 | Lowest — most secure, immutable                 |

### Production rule

Always pin to @v4 or @v4.1.0 in production. Never use @master — I've seen pipelines break at 2 AM because an action maintainer pushed a breaking change to master.

## Chapter 4 — Variables, Secrets, and Contexts

---

This is where most beginners get confused. There are four ways to inject values into a workflow. Each has a purpose.

### 1. env — Environment Variables

```
# Top-level env - available to ALL jobs and steps
env:
  IMAGE_NAME: myapp
  NODE_ENV: production

jobs:
  build:
    # Job-level env - only this job
    env:
      BUILD_MODE: release
    steps:
      # Step-level env - only this step
      - name: Print vars
        env:
          STEP_VAR: hello
        run: |
          echo $IMAGE_NAME      # from top level
          echo $BUILD_MODE      # from job level
          echo $STEP_VAR        # from step level
```

### 2. Secrets — Sensitive Values

Secrets are stored encrypted in GitHub. They NEVER appear in logs — GitHub automatically redacts them. Use for passwords, tokens, API keys.

```
# Store in: Settings -> Secrets and variables -> Actions -> Secrets

- name: Login to DockerHub
  uses: docker/login-action@v3
  with:
    username: ${ secrets.DOCKERHUB_USERNAME }
    password: ${ secrets.DOCKERHUB_TOKEN }
```

### 3. vars — Repository Variables

Non-sensitive config that you want to manage in GitHub UI instead of hardcoding in YAML.

```
# Store in: Settings -> Secrets and variables -> Actions -> Variables
```

```
env:
  REGION: ${vars.AWS_REGION}      # non-sensitive config
  IMAGE_NAME: ${vars.DOCKER_REPO} # repo-level variable
```

## 4. Contexts — Built-in GitHub Information

GitHub automatically provides these — no setup needed. They give you information about the current run, repo, and actor.

| Context                 | Example                         | What it gives you                     |
|-------------------------|---------------------------------|---------------------------------------|
| github.sha              | \${{ github.sha }}              | Full commit hash —<br>abc123def456... |
| github.ref              | \${{ github.ref }}              | refs/heads/main                       |
| github.actor            | \${{ github.actor }}            | Username who triggered the<br>run     |
| github.repository       | \${{ github.repository }}       | owner/repo-name                       |
| github.repository_owner | \${{ github.repository_owner }} | Just the owner/username               |
| github.run_number       | \${{ github.run_number }}       | Sequential run number 1, 2,<br>3...   |
| github.event_name       | \${{ github.event_name }}       | push, pull_request, schedule...       |

## Setting Variables Dynamically During a Run

Sometimes you need to set a variable in one step and use it in the next. You can't use 'export' — each step runs in a fresh shell. Use GITHUB\_ENV instead:

```
- name: Set image tag
  run: |
    SHORT_SHA=$(echo ${github.sha} | cut -c1-8)
    echo "SHORT_SHA=$SHORT_SHA" >> $GITHUB_ENV

- name: Use it in next step
  run: |
    echo "Image tag is: $SHORT_SHA" # works because of GITHUB_ENV
```

### Why not export?

Each step in GitHub Actions runs in a separate shell process. When that process exits, its environment is gone. GITHUB\_ENV is a file that GitHub reads between steps and re-injects into the next step's environment.

## Chapter 5 — Permissions (The Thing Nobody Reads Until It Breaks)

Every workflow run gets a `GITHUB_TOKEN` automatically. This is a short-lived token that lets the workflow interact with GitHub APIs — creating comments, uploading artifacts, reading code, etc.

By default this token has `READ` permission for most things. If your step needs to `WRITE` something (like uploading SARIF to the Security tab), you have to explicitly grant it.

### How to Set Permissions

```
# Option 1 — Top level (all jobs get these permissions)
permissions:
  security-events: write
  contents: read
  actions: read

# Option 2 — Job level (only this job gets write permission) — PREFERRED
jobs:
  security-scan:
    runs-on: ubuntu-latest
    permissions:
      security-events: write    # upload SARIF to Security tab
      contents: read
      actions: read
    steps: [ ... ]
```

| Permission                          | What it allows                    | Common use case                           |
|-------------------------------------|-----------------------------------|-------------------------------------------|
| <code>contents: read</code>         | Read repo files                   | Default for checkout                      |
| <code>contents: write</code>        | Push commits, create releases     | Auto-tagging, changelog generation        |
| <code>security-events: write</code> | Upload SARIF to Security tab      | Trivy, CodeQL scans                       |
| <code>packages: write</code>        | Push to GitHub Container Registry | Pushing Docker images to ghcr.io          |
| <code>pull-requests: write</code>   | Comment on PRs, approve           | Bot comments with test results            |
| <code>id-token: write</code>        | Get OIDC token for cloud auth     | Authenticate to AWS/Azure without secrets |

#### Job-level vs top-level

Always prefer job-level permissions. It's the principle of least privilege — only the job that uploads SARIF needs `security-events:write`. Your test job doesn't need it. Keep permissions as narrow as possible.

## Chapter 6 — Conditional Execution: if, always(), and Failure Handling

### The 'if' Condition

Every step and job can have an 'if' condition. If it evaluates to false, that step is skipped.

```
steps:
  # Run only on main branch
  - name: Deploy to production
    if: github.ref == 'refs/heads/main'
    run: ./deploy.sh

  # Run only on pull requests
  - name: Post PR comment
    if: github.event_name == 'pull_request'
    run: echo 'This is a PR run'

  # Run only if a previous step failed
  - name: Send alert on failure
    if: failure()
    run: curl -X POST $SLACK_WEBHOOK -d '{"text":"Build failed!"}'

  # Run even if a previous step failed
  - name: Upload SARIF report
    if: always()
    uses: github/codeql-action/upload-sarif@v4
```

### Status Functions You Need to Know

| Function    | When it runs                                | Use case                          |
|-------------|---------------------------------------------|-----------------------------------|
| success()   | Only if all previous steps passed (DEFAULT) | Normal steps                      |
| failure()   | Only if a previous step failed              | Alerting, cleanup on failure      |
| always()    | No matter what happened                     | Upload reports, cleanup resources |
| cancelled() | Only if the run was manually cancelled      | Rare — cleanup on cancel          |

#### The if: always() trap

In your Trivy workflow, the SARIF upload had 'if: always()'. This is correct — you want the report uploaded even if Trivy found vulnerabilities (which causes exit code 1). But if the scan step failed for a different reason (like image not found), 'if: always()' meant the upload ran anyway — with no file to upload. Always make sure the file actually exists before uploading.



## Chapter 7 — Artifacts and Caching

---

### Artifacts — Sharing Files Between Jobs

Remember — each job runs on a separate VM. If Job 1 builds something and Job 2 needs it, you must upload it as an artifact in Job 1 and download it in Job 2.

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Build app
        run: npm run build

      # Upload the built files as an artifact
      - name: Upload build artifact
        uses: actions/upload-artifact@v4
        with:
          name: build-output          # artifact name
          path: dist/                 # what to upload
          retention-days: 7           # how long to keep it

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      # Download the artifact from the build job
      - name: Download build artifact
        uses: actions/download-artifact@v4
        with:
          name: build-output
          path: dist/

      - name: Deploy
        run: ./deploy.sh dist/
```

### Caching — Speed Up Repeated Runs

Artifacts are for sharing files between jobs in the same run. Cache is for reusing files between different runs — like `node_modules` or `pip` packages.

```
- name: Cache node_modules
  uses: actions/cache@v4
  with:
    path: node_modules
    key: ${{ runner.os }}-npm-${{ hashFiles('package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-npm-
```

```
- name: Install dependencies
  run: npm install

# OR - use cache built into setup-node (simpler):
- uses: actions/setup-node@v4
  with:
    node-version: '20'
    cache: 'npm' # automatic cache
    cache-dependency-path: app/package.json
```

### Cache key strategy

The cache key uses `hashFiles('package-lock.json')`. If the lockfile changes (new packages added), the hash changes, the old cache is missed, fresh install happens. This is exactly what you want — don't cache stale dependencies.

## Chapter 8 — Debugging Failed Workflows

---

This is where experience really shows. Here are the debugging moves I use in order.

### Step 1 — Read the Actual Error Line

Click on the failed step in the GitHub Actions UI. Read the LAST red line before the exit code. That's the real error. Everything above it is context.

### Step 2 — Add Debug Echo Steps

```
- name: Debug - what do we have?
  run: |
    echo "=== Environment ==="
    env | sort
    echo "=== Files ==="
    ls -la
    echo "=== Docker images ==="
    docker images
    echo "=== Working directory ==="
    pwd
```

### Step 3 — Enable Runner Debug Logging

Go to Settings -> Secrets -> add these secrets:

```
ACTIONS_RUNNER_DEBUG = true
ACTIONS_STEP_DEBUG = true
```

Re-run the workflow. You'll get extremely verbose logs showing every action GitHub Actions takes internally.

### Step 4 — Re-run with Debug

In the GitHub Actions UI, click 'Re-run jobs' -> 'Re-run all jobs' -> check 'Enable debug logging'. No secret needed.

### Common Errors and What They Actually Mean

| Error message                          | Real cause                    | Fix                          |
|----------------------------------------|-------------------------------|------------------------------|
| Resource not accessible by integration | GITHUB_TOKEN lacks permission | Add permissions block to job |

| Error message                                          | Real cause                                    | Fix                                        |
|--------------------------------------------------------|-----------------------------------------------|--------------------------------------------|
| Path does not exist: file.sarif                        | Previous step failed before creating the file | Check the step above it in logs            |
| 401 Unauthorized: access token has insufficient scopes | DockerHub PAT is read-only                    | Regenerate token with Read & Write scope   |
| Repository not found                                   | Repo doesn't exist or token can't access it   | Create repo or fix token permissions       |
| Process completed with exit code 1                     | Your command returned failure                 | Check the lines above — it's not this line |
| Node 20 is being deprecated                            | Action uses old Node runtime                  | Upgrade action version to v4               |

## Chapter 9 — Security Best Practices

---

After 20 years, the security mistakes I see most often in GitHub Actions are surprisingly consistent. Here's how to not make them.

### 1. Never Put Secrets in env at Top Level

```
# BAD — secret available to all jobs including untrusted ones
env:
  DB_PASSWORD: ${ secrets.DB_PASSWORD }

# GOOD — only the step that needs it gets it
- name: Connect to database
  env:
    DB_PASSWORD: ${ secrets.DB_PASSWORD }
  run: ./connect.sh
```

### 2. Pin Action Versions to SHA for Critical Workflows

```
# Good — pinned to major version
uses: actions/checkout@v4

# Better — pinned to exact commit SHA (supply chain attack protection)
uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2
```

### 3. Use OIDC Instead of Long-lived Secrets for Cloud Auth

Instead of storing `AWS_ACCESS_KEY` or Azure credentials as secrets (which can be stolen), use OIDC — a temporary token that expires after the job.

```
permissions:
  id-token: write      # required for OIDC
  contents: read

- name: Authenticate to Azure
  uses: azure/login@v2
  with:
    client-id: ${ secrets.AZURE_CLIENT_ID }
    tenant-id: ${ secrets.AZURE_TENANT_ID }
    subscription-id: ${ secrets.AZURE_SUBSCRIPTION_ID }
    # No password/secret needed — OIDC handles it
```

### 4. Limit What Triggers Can Do on Forks

```
# PRs from forks can't access secrets by default - good
# But be careful with pull_request_target - it CAN access secrets
# Only use pull_request_target when you explicitly need fork PR access

on:
  pull_request:          # safe - no secret access for forks
    branches: [ main ]
```

### The fork + secret problem

This is exactly what your SARIF upload warning was about — 'Action is running on a pull request from a fork'. Fork PRs don't get GITHUB\_TOKEN write access by design. It's a security boundary. Your fix (adding permissions block) works for non-fork PRs. For fork PRs, you need `pull_request_target` which is more complex.

## Chapter 10 — Real World Patterns

---

### Pattern 1 — The GitOps Update (What You're Doing)

Push code -> build image -> update manifest repo -> ArgoCD deploys. This is GitOps.

```
- name: Update manifest repo
  run: |
    git config --global user.email 'ci-bot@github.com'
    git config --global user.name 'CI Bot'

    git clone https://x-access-token:${{ secrets.MANIFEST_REPO_TOKEN
    }}@github.com/\
      ${{{ github.repository_owner }}/nodejs-manifests.git

    cd nodejs-manifests

    # Update image tag in deployment.yaml
    sed -i 's|image: .*/nodejs-app:.*|image: ${{{ env.IMAGE_NAME }}}:${{{
    github.sha }}}|g' \
      deployment.yaml

    git add deployment.yaml
    git commit -m 'ci: update image to ${{{ github.sha }}} [skip ci]'
    git push

    # [skip ci] prevents infinite loop - ArgoCD push triggers CI, which would push
    again
```

### Pattern 2 — Matrix Builds (Test Multiple Versions)

```
jobs:
  test:
    strategy:
      matrix:
        node-version: ['18', '20', '22']    # runs 3 parallel jobs
        os: [ubuntu-latest, windows-latest] # runs on 2 OS = 6 total jobs
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
      - run: npm test
```

### Pattern 3 — Reusable Workflows

Instead of copy-pasting the same deploy job into 5 repos, define it once and call it from anywhere.

```
# In .github/workflows/deploy.yml (the reusable workflow)
on:
  workflow_call:          # makes it callable
  inputs:
    environment:
      required: true
      type: string

# In another repo's workflow:
jobs:
  deploy:
    uses: myorg/shared-workflows/.github/workflows/deploy.yml@main
    with:
      environment: production
    secrets: inherit       # pass all secrets through
```

## Chapter 11 — Your Current Pipeline, Explained

---

Let's walk through exactly what your CI pipeline does and why each decision was made.

### Job 1 — Unit Tests

| Step                             | Why it's there                                            |
|----------------------------------|-----------------------------------------------------------|
| actions/checkout@v4              | Clone your repo onto the runner — always first            |
| actions/setup-node@v4 with cache | Install Node 20, cache node_modules for speed             |
| working-directory: app           | Your package.json is in app/ subfolder, not root          |
| npm install                      | Install dependencies fresh each run                       |
| npm test                         | Run Jest — exit code 1 if any test fails, killing the job |
| upload-artifact: coverage-report | Save coverage HTML so you can view it later in GitHub UI  |

### Job 2 — Vulnerability Scan

| Step                                | Why it's there                                         |
|-------------------------------------|--------------------------------------------------------|
| needs: test                         | Don't waste time scanning if tests fail                |
| permissions: security-events: write | Required to upload SARIF to GitHub Security tab        |
| docker build with scan- tag         | Build the image to scan it before pushing to DockerHub |
| trivy format: table, exit-code: 1   | Human readable output, FAIL if CRITICAL/HIGH found     |
| trivy format: sarif, if: always()   | Always save SARIF even if scan found vulns (exit 1)    |
| upload-sarif, if: always()          | Always upload report to Security tab                   |

### Job 3 — Build and Push

| Step                              | Why it's there                                      |
|-----------------------------------|-----------------------------------------------------|
| needs: [test, vulnerability-scan] | Only push clean, tested, scanned images             |
| docker/setup-buildx-action        | Enable BuildKit — faster builds with better caching |

| Step                                 | Why it's there                               |
|--------------------------------------|----------------------------------------------|
| docker/login-action                  | Authenticate to DockerHub before pushing     |
| build-push-action with two tags      | Push :latest AND :sha — SHA for traceability |
| cache-from/cache-to: type=gha        | Cache Docker layers in GitHub Actions cache  |
| git clone manifest repo + sed + push | GitOps — update what ArgoCD should deploy    |

### Why two Docker tags?

'latest' is convenient — anyone can pull the most recent image without knowing the SHA. 'sha' is the real tag — it's immutable, traceable, and is what your manifest repo references. If something goes wrong in production, you look at the manifest, see the SHA, and know exactly which commit caused it.

## Chapter 12 — Interview Questions on GitHub Actions

---

These are the questions you'll get in DevOps and SRE interviews. Answer them with your actual experience.

### Q1: What is the difference between a job and a step?

#### Strong answer

A job is a unit of work that runs on its own virtual machine. Steps are sequential tasks inside a job that share the same machine and filesystem. Jobs run in parallel by default — if you need them in sequence, use 'needs'. Steps within a job always run in order. In my pipeline, I have three jobs: test, vulnerability-scan, and build-and-push. They run on separate VMs. The steps inside each job share that VM's environment.

### Q2: How do you pass data between jobs?

#### Strong answer

Since each job runs on a different VM, you can't share files directly. You use artifacts — upload-artifact in Job 1, download-artifact in Job 2. For simple values like version numbers or booleans, you use job outputs: set the output with `echo 'name=value' >> $GITHUB_OUTPUT`, then reference it in the next job with `needs.job-name.outputs.name`. In my pipeline, I upload the coverage report as an artifact so it's accessible after the run.

### Q3: How do you secure secrets in GitHub Actions?

#### Strong answer

GitHub automatically redacts secret values from logs. I store sensitive values in Settings -> Secrets -> Actions, never hardcode them in YAML. I use them with `${{ secrets.NAME }}` syntax. For cloud authentication, I prefer OIDC over long-lived credentials — it gives a temporary token scoped to that specific job run. I also scope permissions at job level using the permissions block rather than giving all jobs broad access.

### Q4: What is GITHUB\_TOKEN and how is it different from a PAT?

#### Strong answer

GITHUB\_TOKEN is automatically generated by GitHub for each workflow run. It expires when the job ends and has permissions scoped to the current repository. A PAT (Personal Access Token) is created manually, doesn't expire unless you set it to, and can have access to multiple repos or your entire account. I use GITHUB\_TOKEN for things like uploading artifacts and SARIF reports. I use PATs stored as secrets when I need to access a different repo — like when my CI pipeline updates my manifest repo for GitOps.

## Q5: How would you reduce CI pipeline duration?

### Strong answer

Four main levers: caching, parallelism, skipping unnecessary work, and optimizing the steps themselves. I use setup-node with cache: npm to avoid reinstalling packages each run. Jobs that don't depend on each other run in parallel. I use paths filtering on triggers so pushes to docs/ don't trigger the full build. For Docker, I use BuildKit with GitHub Actions cache (cache-from: type=gha) to reuse unchanged layers. In my current pipeline, caching alone saved about 2 minutes per run.

## Chapter 13 — What to Learn Next

You've got a solid production pipeline running. Here's what I'd add in order of usefulness for your career:

| Topic                        | Why it matters                                                        | Priority |
|------------------------------|-----------------------------------------------------------------------|----------|
| OIDC authentication to Azure | No long-lived Azure creds in secrets — uses your AKS/Azure background | High     |
| Self-hosted runners          | Run pipelines on your own infra — common in enterprise AKS setups     | High     |
| Reusable workflows           | DRY across multiple repos — asked in every senior interview           | High     |
| Environment protection rules | Require manual approval before deploying to production                | Medium   |
| Matrix builds                | Test multiple Node/Python versions in parallel                        | Medium   |
| GitHub Packages (ghcr.io)    | Push Docker images to GitHub instead of DockerHub                     | Medium   |
| Dependabot for Actions       | Auto-update action versions — security hygiene                        | Low      |
| Act — run Actions locally    | Test workflows without pushing to GitHub — huge time saver            | Low      |

## The Single Best Thing You Can Do Right Now

### Advice

Your existing pipeline is your best learning tool. Add one new feature to it every week. This week: add OIDC auth to Azure instead of storing credentials. Next week: add a manual approval step before the manifest update. The week after: add a Slack notification on failure. Real pipelines teach faster than any course.

## Useful References

- GitHub Actions official docs: [docs.github.com/en/actions](https://docs.github.com/en/actions)
- GitHub Actions Marketplace: [github.com/marketplace?type=actions](https://github.com/marketplace?type=actions)
- Aqua Trivy action: [github.com/aquasecurity/trivy-action](https://github.com/aquasecurity/trivy-action)
- Act (run locally): [github.com/nektos/act](https://github.com/nektos/act)
- Your repo: [github.com/mohan6451/terraform-devops-portfolio](https://github.com/mohan6451/terraform-devops-portfolio)

---

*You're not learning GitHub Actions. You're already using it in production.*

*This guide just explains what you're already doing — and why.*